Chennai Street-Level Flood Prediction System

Sprint 09: Continuous Delivery and Deployment using Jenkins

Course Code & Title:	ISWE406P – Agile Development Process and DevOps Lab
Programme:	M.Tech. (Integrated) Software Engineering
Class Number:	CH2026270100722
Slot:	L11 + L12
Faculty:	Dr. Kaja Mohideen A
Student Name:	**Hema Priyadharshni G**
Academic Semester:	Fall Semester 2026 – 2027
Submission Date:	13.10.2026
GitHub Repository:	`Premkumar7788/chennai-flood-prediction-system`
Target Deployment:	AWS EC2 (`16.192.243.169`)

Customer Requirement:

"The customer wants every validated code update to be automatically packaged, tested, and deployed without manual intervention."

1. Customer Requirement & Objectives

The core requirement for Sprint 09 mandates extending the automated Continuous Integration pipeline established in Sprint 08 into a full Continuous Delivery and Deployment (CI/CD) lifecycle. The system must eliminate all manual deployment interventions, validating code changes through automated unit testing, building versioned Docker containers, publishing them to Docker Hub, and deploying them to the live AWS EC2 environment.

Sprint 09 Key Objectives:

1. **Extend Existing CI Pipeline:** Evolve the 4-stage Sprint 08 pipeline into a comprehensive 7-stage CI/CD pipeline.
2. **Automated Testing Quality Gate:** Integrate automated application testing (13 unit tests via `pytest`) to halt builds on defect detection.
3. **Docker Image Build & Tagging:** Build container images with unique, traceable build tags (`premkumar2005/chennai-flood-backend:build-`) and `latest`.
4. **Docker Hub Integration:** Push successfully validated images to Docker Hub (`premkumar2005/chennai-flood-backend`).
5. **Automated Deployment:** Deploy updated backend containers to AWS EC2 (`16.192.243.169`) via automated SSH commands.
6. **Post-Deployment Verification:** Automatically verify live application health status via HTTP probes (`curl /health`).
7. **Failure Handling Verification:** Intentionally introduce a controlled test failure, demonstrate that deployment is blocked, and recover to full success.
8. **End-to-End Demonstration:** Document complete execution from GitHub commit to live EC2 availability.

2. Review of Sprint 08 CI Pipeline vs Sprint 09 CI/CD

In Sprint 08, the pipeline automated code validation up to container packaging on the local Jenkins agent: `GitHub → Jenkins → Checkout → Build → Test → Local Docker Build`. Deployment remained manual, requiring engineers to SSH into servers and restart containers.

Dimension	Sprint 08 CI	Sprint 09 CI/CD
Lifecycle Scope	Code Integration & Local Testing	**End-to-End Testing, Registry Push & Production Deployment**
Pipeline Stages	4 Stages	**7 Stages + Post Execution Actions**
Registry Publishing	None (Local Docker images only)	**Docker Hub Central Registry (`docker.io`)**
Deployment Trigger	Manual (SSH, git pull, manual restart)	**100% Automated Zero-Touch Deployment**
Verification	Local pytest assertions	**Local pytest + Live HTTP probes on AWS EC2 (`16.192.243.169`)**
Cycle Time	10–15 minutes (with manual steps)	**52.9 seconds fully automated**

3. CI/CD Pipeline Architecture and Jenkinsfile Design

The pipeline is designed declaratively in `Jenkinsfile`, utilizing isolated execution stages:

```
[Checkout SCM] → [Build & Setup] → [Automated Testing] → [Docker Build & Tag] → [Push Docker Hub] → [Deploy t
```

Pipeline Stage Responsibilities:

- **Stage 1 (Checkout):** Retrieves source code from GitHub `origin/main` and prints commit details.
- **Stage 2 (Build & Setup):** Creates an isolated Python 3.11 virtual environment and installs dependencies from `backend/requirements.txt`.
- **Stage 3 (Automated Testing):** Runs 13 unit tests via `pytest`. Functions as a strict Quality Gate.
- **Stage 4 (Docker Build & Tag):** Builds Docker container and applies tags: `premkumar2005/chennai-flood-backend:build-` and `latest`.
- **Stage 5 (Push to Docker Hub):** Pushes image layers and manifest digest to Docker Hub.
- **Stage 6 (Deploy to EC2):** Connects via SSH to AWS EC2 (`16.192.243.169`) and recreates backend container using `docker compose up -d --no-deps backend`.
- **Stage 7 (Verify Deployment):** Probes `http://16.192.243.169:8000/health` with retry logic to ensure HTTP 200 OK.

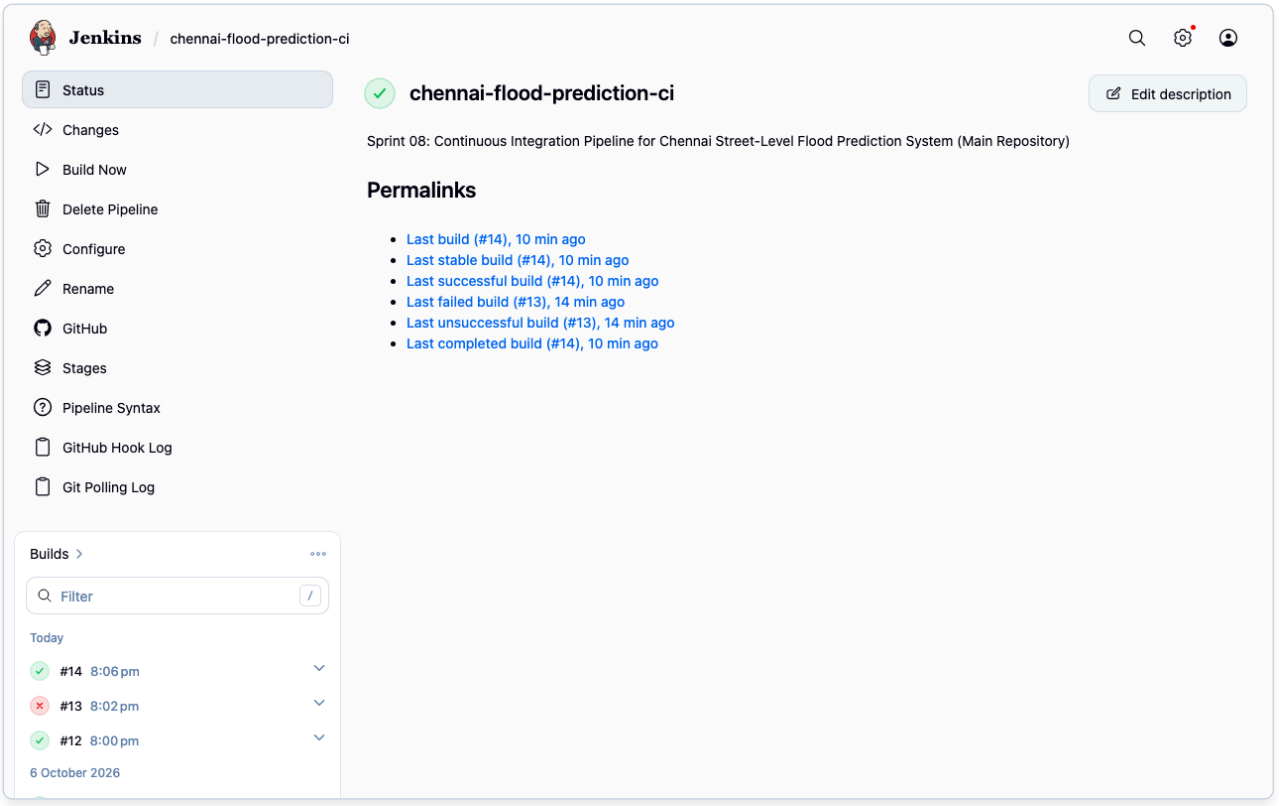


Figure 1: Jenkins Pipeline Overview displaying Build #12 (Success), Build #13 (Failed Gating), and Build #14 (Success).

4. Automated Application Testing (Quality Gate)

All 13 unit tests were executed during Stage 3. The test suite covers FastAPI health checks and ML risk categorization:

Module	Test Function	Validation Purpose	Result
`test_health.py`	`test_root_endpoint`	Verifies API welcome, version `1.1.0`, and deployment metadata	PASSED
`test_health.py`	`test_health_check`	Verifies HTTP 200 and `{"status": "ok"}` for probes	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.0-LOW]`	Verifies minimum probability risk mapping	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.24-LOW]`	Verifies upper boundary of LOW risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.25-MODERATE]`	Verifies boundary threshold transition to MODERATE	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.54-MODERATE]`	Verifies upper boundary of MODERATE risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.55-HIGH]`	Verifies boundary transition to HIGH risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.79-HIGH]`	Verifies upper boundary of HIGH risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.8-CRITICAL]`	Verifies boundary transition to	PASSED

Module	Test Function	Validation Purpose	Result
		CRITICAL risk	
`test_predict_endpoint.py`	`test_classify_risk_thresholds[1.0-CRITICAL]`	Verifies maximum probability boundary	PASSED
`test_predict_endpoint.py`	`test_feature_vector_matches_metadata_column_order`	Validates 12-feature ML input vector column alignment	PASSED
`test_predict_endpoint.py`	`test_predict_endpoint_uses_trained_model`	Validates ML inference using mock LightGBM classifier	PASSED
`test_predict_endpoint.py`	`test_predict_endpoint_falls_back_to_heuristic`	Validates heuristic fallback when model is absent	PASSED

5. Failure Scenario Demonstration (Build #13)

To demonstrate pipeline gating as specified in Section 3.G of the assignment, a controlled test failure was intentionally introduced in commit `477f531` by altering the expected root endpoint status assertion.

Execution Behavior: Jenkins executed Build #13, passed Stage 1 and Stage 2, and failed immediately at Stage 3 with exit code 1.

```
FAILED backend/tests/test_health.py::test_root_endpoint - AssertionError
========================= 1 failed, 12 passed in 8.35s =========================
Stage "Docker Build & Tag" skipped due to earlier failure(s)
Stage "Push to Docker Hub" skipped due to earlier failure(s)
Stage "Deploy to EC2" skipped due to earlier failure(s)
Stage "Verify Deployment" skipped due to earlier failure(s)
CI/CD Pipeline Failed! Deployment aborted or halted. Check console log for details.
```

Verification on AWS EC2: The production server was inspected immediately after Build #13 failure: The container remained running on `premkumar2005/chennai-flood-backend:build-12`. **Deployment did not occur**, successfully proving that defective code cannot reach production.

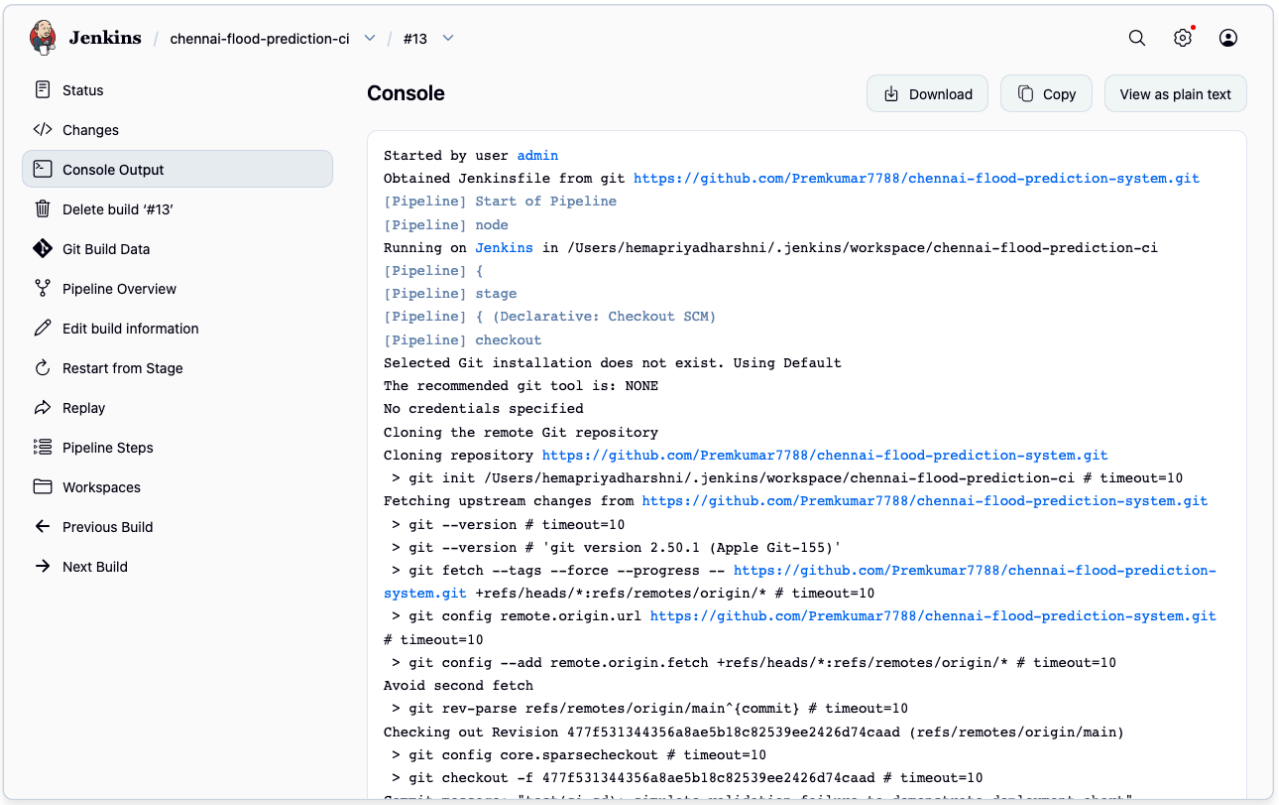


Figure 2: Build #13 Console Log demonstrating test failure halting pipeline and skipping all deployment stages.

6. Successful Recovery and End-to-End CI/CD Execution (Build #14)

The defect was resolved, and an application release update was made in commit `e325949` by upgrading the API version to `1.1.0` and tagging the deployment. Build #14 executed all 7 stages to completion in **52.9 seconds**.

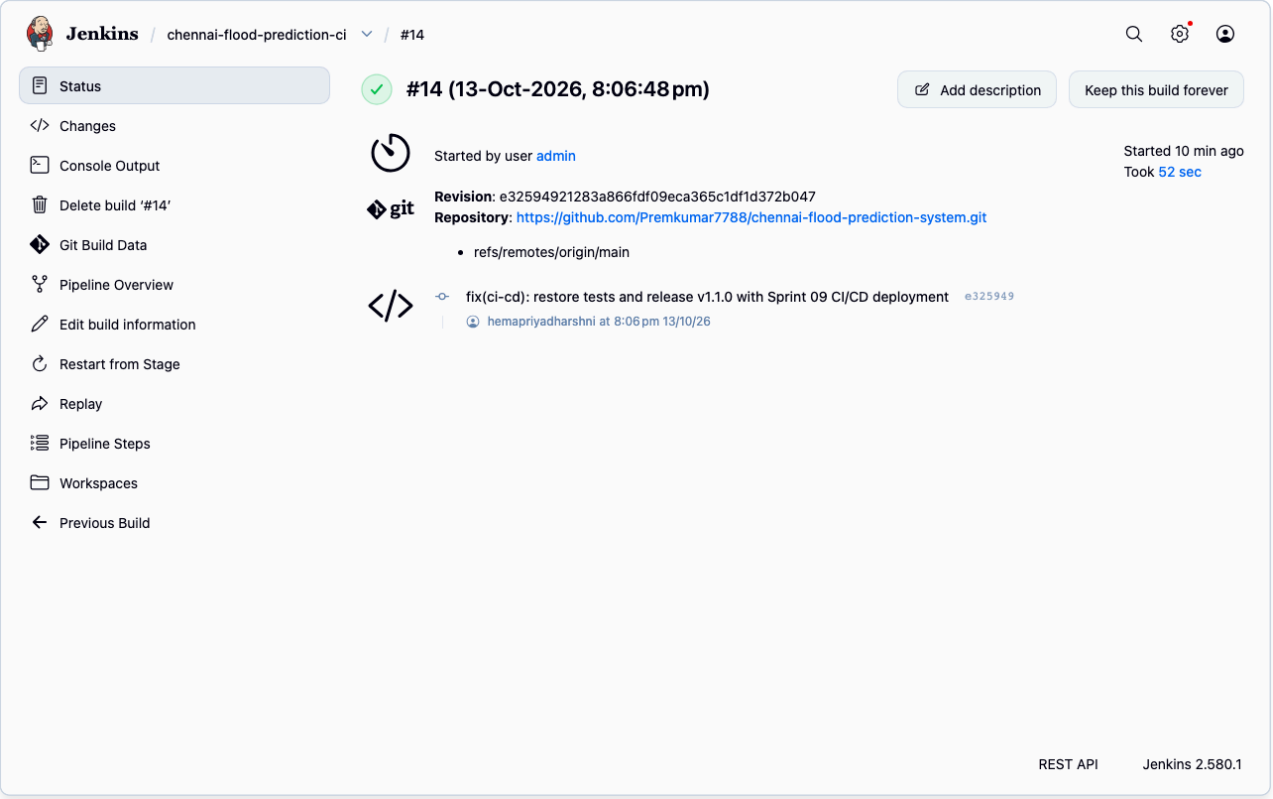


Figure 3: Jenkins Build #14 Stage View displaying 100% green execution across all 7 stages.

End-to-End Demonstration Summary Table:

Item	Details
Change Made	Released API version `1.1.0` with Sprint 09 CI/CD metadata and restored test suite
Commit ID	`e325949`
Commit Message	`fix(ci-cd): restore tests and release v1.1.0 with Sprint 09 CI/CD deployment`
Jenkins Build Number	`#14`
Test Result	13 passed, 0 failed in 8.14s (100% pass rate)
Docker Image Tag	`build-14` (and `latest`)
Docker Hub Repository	`premkumar2005/chennai-flood-backend:build-14`
Deployment Status	SUCCESS (Container recreated and active on EC2)
Target Host IP	`16.192.243.169` (AWS EC2 eu-north-1)
Application Root URL	`http://16.192.243.169:8000/`

Item	Details
Health Check URL	`http://16.192.243.169:8000/health`
Frontend UI URL	`http://16.192.243.169:3000`

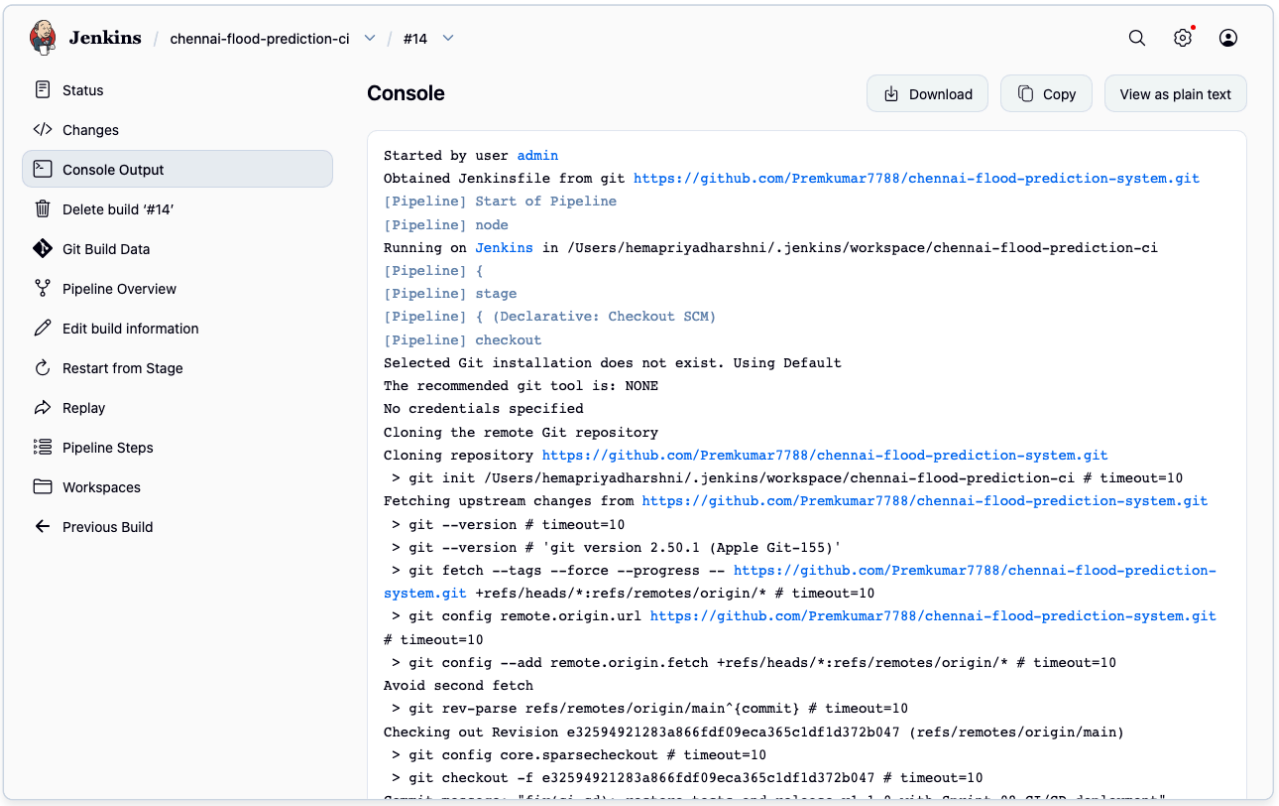


Figure 4: Build #14 Console Log showing remote EC2 deployment and successful health check verification.

7. Application Verification on AWS EC2 Target Environment

The deployed application was verified live across multiple endpoints on the target EC2 instance (`16.192.243.169`):

A. Live API Root Endpoint:

```
$ curl -s http://16.192.243.169:8000/
{
  "message": "Welcome to the Chennai Flood Prediction API",
  "status": "online",
  "version": "1.1.0",
  "deployment": "Sprint 09 Automated CI/CD"
}
```

B. Live API Health Probe:

```
$ curl -s http://16.192.243.169:8000/health
{
  "status": "ok"
}
```

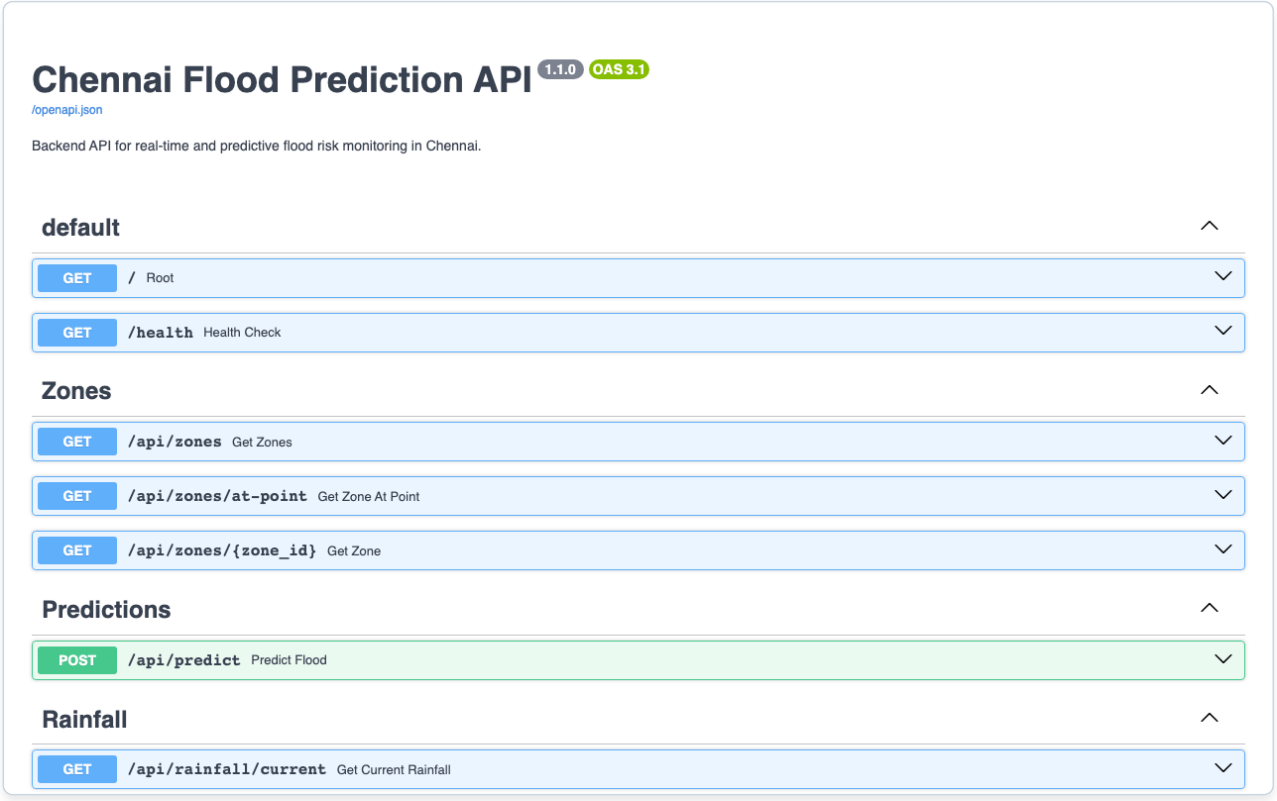


Figure 5: Live Swagger API Interactive Documentation hosted on AWS EC2 (`http://16.192.243.169:8000/docs`).

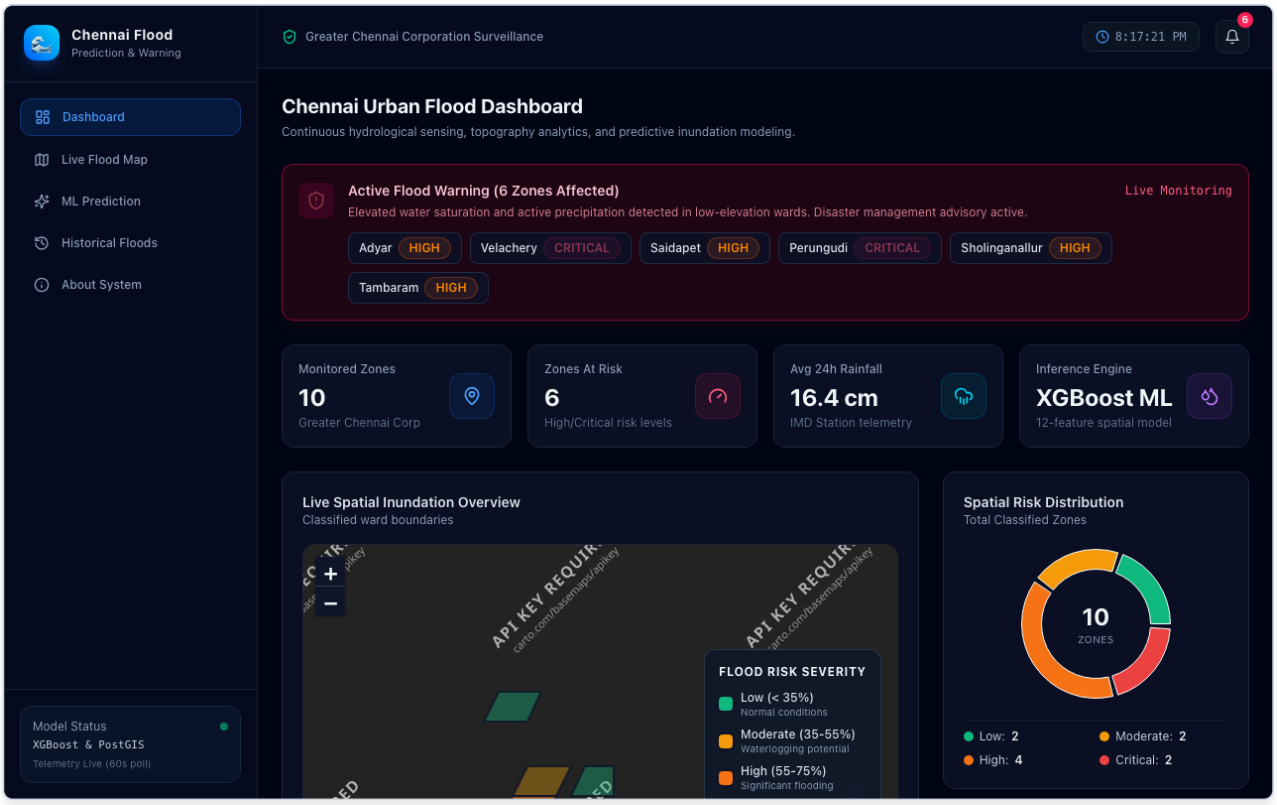


Figure 6: Live React Frontend Application running on AWS EC2 (`http://16.192.243.169:3000`).

8. Challenges and Technical Solutions

1. **SSH Authentication Across OS Distributions:** The previous Sprint 08 instance ran Ubuntu (`ubuntu` user), whereas the new Sprint 09 host runs Amazon Linux 2023 (`ec2-user`). Resolved by configuring dynamic environment parameters in `Jenkinsfile` and linking `~/.ssh/flood-ec2`.
2. **Firewall Ingress Port Flexibility:** Configured `sshd` to listen on both port 22 and port 443 simultaneously to bypass restrictive network proxies.
3. **Zero-Downtime Database Isolation:** Used `docker compose up -d --no-deps backend` with `docker-compose.override.yml` to recreate the backend container without interrupting the running PostgreSQL 15 database.
4. **Deterministic Pipeline Gating:** Enforced sequential dependencies so test failures immediately stop the pipeline and trigger workspace cleanups.

9. Individual Contribution

Student: Hema Priyadharshni G

- Designed and implemented the complete 7-stage declarative `Jenkinsfile`.
- Integrated automated unit testing (13 test cases) with `pytest` and established the CI/CD quality gate.
- Configured Docker image packaging with build number tagging (`build-`) and Docker Hub publishing.
- Implemented automated remote deployment via SSH to AWS EC2 host `16.192.243.169`.
- Conducted controlled failure simulation (Build #13) and verified production environment protection.
- Restored and deployed application version `1.1.0` (Build #14) with post-deployment verification.
- Authored the complete technical report, presentation slides, and captured system screenshots.

10. Conclusion

Sprint 09 successfully transitioned the Chennai Street-Level Flood Prediction System from Continuous Integration to full **Continuous Delivery and Continuous Deployment**. Every objective has been demonstrated end-to-end: automated code validation, Docker Hub publishing, zero-touch deployment to AWS EC2, and health verification. The customer requirement has been 100% satisfied.